

UACM

Universidad Autónoma
de la Ciudad de México

Nada humano me es ajeno

Academia de Informática Cuauhtepc Matutino

Ingeniería de Software

Construcción y evolución del software

Laboratorio de Ingeniería de Software

Profesor: Emmanuel Vite Sevilla

Nombre de la práctica: Programa uno

Número: 3

Alumno(s)/Equipo:

Rios Sánchez Axel Javier

Maldonado Espinosa Evelin

Suarez López Jesús Josué

Chavez Galindo Mauricio Charbel

Fecha: 14/04/2026



Introducción:

La transición del simulador de UNO desde su segunda versión hacia la tercera representa un salto cualitativo fundamental en la arquitectura del software, marcando la clara evolución de un código procedimental y monolítico hacia un diseño puramente orientado a objetos. Este proceso de refactorización estructural se centró en aplicar principios de diseño profesional, desarmando un sistema rígido y acoplado para dar paso a un entorno donde el polimorfismo, la herencia y la estricta separación de responsabilidades toman el control. El resultado de esta transformación es una base de código limpia e intuitiva que deja atrás las limitaciones de la programación estructurada básica para adoptar los estándares de calidad y las buenas prácticas que rigen la industria actual.

Evidencia código versión 2

```

class Carta {
    String color;
    String valor;

    static final String RESET = "\u001B[0m";
    static final String ROJO = "\u001B[31m";
    static final String VERDE = "\u001B[32m";
    static final String AMARILLO = "\u001B[33m";
    static final String AZUL = "\u001B[34m";
    static final String BLANCO = "\u001B[37m";

    public Carta(String color, String valor) {
        this.color = color;
        this.valor = valor;
    }

    String colorANSI() {
        switch (color) {
            case "Rojo": return ROJO;
            case "Verde": return VERDE;
            case "Azul": return AZUL;
            case "Amarillo": return AMARILLO;
            default: return BLANCO;
        }
    }
}

```

```

    public String[] dibujar() {
        String c = colorANSI();
        return new String[]{
            c + "+-----+" + RESET,
            c + "| " + String.format("%-8s", valor) + " |" + RESET,
            c + "| " + RESET,
            c + "| " + String.format("%-8s", color) + " |" + RESET,
            c + "| " + RESET,
            c + "| " + String.format("%8s", valor) + " |" + RESET,
            c + "+-----+" + RESET
        };
    }
}

```

Evidencia del código versión 3

```
protected String getANSI() {  
    return switch (color) {  
        case "Rojo" -> ROJO;  
        case "Verde" -> VERDE;  
        case "Azul" -> AZUL;  
        case "Amarillo" -> AMARILLO;  
        default -> BLANCO;  
    };  
}  
  
public String getLinea(int linea) {  
    String c = getANSI();  
    String vDibujo = valor.equals(anObject: "Color") ? "comodin" : valor;  
    String[] diseño = {  
        c + "+-----+" + RESET,  
        c + "| " + String.format(format: "%-8s", vDibujo) + " |" + RESET,  
        c + "|          |" + RESET,  
        c + "| " + String.format(format: "%-8s", color) + " |" + RESET,  
        c + "|          |" + RESET,  
        c + "| " + String.format(format: "%-8s", vDibujo) + " |" + RESET,  
        c + "+-----+" + RESET  
    };  
    return diseño[linea];  
}
```

Universidad Autónoma
de la Ciudad de México
NADA HUMANO ME ES AJENO

```
abstract class Carta {
    protected String color;
    protected String valor;

    public static final String RESET = "\u001B[0m";
    public static final String ROJO = "\u001B[31m";
    public static final String VERDE = "\u001B[32m";
    public static final String AZUL = "\u001B[34m";
    public static final String AMARILLO = "\u001B[33m";
    public static final String BLANCO = "\u001B[37m";

    public Carta(String color, String valor) {
        this.color = color;
        this.valor = valor;
    }

    public String getColor() { return color; }
    public String getValor() { return valor; }

    public abstract boolean esCompatible(Carta otra, String colorActual);

    /**
     * @param turnos Componente que maneja el flujo de turnos
     * @param reglas Componente que impone penalizaciones y efectos
     */
    public abstract void aplicarEfecto(ManejadorTurnos turnos, GestorReglas reglas);
}
```

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO


```

class CartaNumerica extends Carta {
    public CartaNumerica(String color, String valor) { super(color, valor); }

    @Override
    public boolean esCompatible(Carta otra, String colorActual) {
        return this.color.equals(colorActual) || this.valor.equals(otra.getValor());
    }

    @Override
    public void aplicarEfecto(ManejadorTurnos turnos, GestorReglas reglas) {
        // Las cartas numéricas no alteran el flujo del juego
    }
}

```

```

class CartaAccion extends Carta {
    public CartaAccion(String color, String valor) { super(color, valor); }

    @Override
    public boolean esCompatible(Carta otra, String colorActual) {
        return this.color.equals(colorActual) || this.valor.equals(otra.getValor());
    }

    @Override
    public void aplicarEfecto(ManejadorTurnos turnos, GestorReglas reglas) {
        switch (valor) {
            case "Salto" -> turnos.activarSalto();
            case "Rev" -> turnos.invertirDireccion();
            case "+2" -> reglas.penalizarSiguiente(cantidadCartas: 2);
        }
    }
}

```

```

class CartaComodin extends Carta {
    public CartaComodin(String valor) { super(color: "Negro", valor); }

    @Override
    public boolean esCompatible(Carta otra, String colorActual) {
        return true;
    }

    @Override
    public void aplicarEfecto(ManejadorTurnos turnos, GestorReglas reglas) {
        reglas.solicitarCambioColor();
        if (valor.equals(anObject: "+4")) reglas.penalizarSiguiente(cantidadCartas: 4);
    }
}

```

¿Qué se mejoró? Transformaste una clase monolítica rígida (Carta) en una estructura orientada a objetos usando herencia. Ahora tienes una clase abstracta base y subclases especializadas (CartaNumerica, CartaAccion, CartaComodin).

También mejoraste la representación en consola y separaste el motor del juego inyectando dependencias como ManejadorTurnos y GestorReglas.

¿Por qué se hizo? Para aplicar polimorfismo y los principios SOLID (como el de Responsabilidad Única). En la primera versión, habrías necesitado condicionales gigantescos (muchos if o switch) para saber qué efecto aplicar. Ahora, cada carta sabe exactamente cómo comportarse y qué efecto ejecutar por sí misma. Esto desacopla la lógica, haciendo que tu código en Java sea mucho más limpio, modular, escalable y fácil de mantener

Evidencia código versión 2

```
class Baraja {
    List<Carta> cartas = new ArrayList<>();

    public Baraja() {
        crear();
        Collections.shuffle(cartas);
    }

    void crear() {
        String[] colores = {"Rojo", "Verde", "Azul", "Amarillo"};
        for (String color : colores) {
            cartas.add(new Carta(color, "0"));
            for (int i = 1; i <= 9; i++) {
                cartas.add(new Carta(color, "" + i));
                cartas.add(new Carta(color, "" + i));
            }
            // Implementación de cartas especiales: Reversa, Salto, Roba 2
            for (int i = 0; i < 2; i++) {
                cartas.add(new Carta(color, "+2"));
                cartas.add(new Carta(color, "salto"));
                cartas.add(new Carta(color, "rev"));
            }
        }
        // Implementación de cartas especiales: Roba 4 y Comodín
        for (int i = 0; i < 4; i++) {
            cartas.add(new Carta("Negro", "+4"));
            cartas.add(new Carta("Negro", "comodín"));
        }
    }

    Carta repartir() {
        if (cartas.isEmpty()) return null;
        return cartas.remove(cartas.size() - 1);
    }
}
```

Evidencia código versión 3

```

class Baraja {
    private LinkedList<Carta> mazo = new LinkedList<>();
    private Stack<Carta> descarte = new Stack<>();

    public Baraja() {
        String[] colores = {"Rojo", "Verde", "Azul", "Amarillo"};
        for (String c : colores) {
            mazo.add(new CartaNumerica(c, valor: "0"));
            for (int i = 1; i <= 9; i++) {
                mazo.add(new CartaNumerica(c, String.valueOf(i)));
                mazo.add(new CartaNumerica(c, String.valueOf(i)));
            }
            for (int i = 0; i < 2; i++) {
                mazo.add(new CartaAccion(c, valor: "Salto"));
                mazo.add(new CartaAccion(c, valor: "Rev"));
                mazo.add(new CartaAccion(c, valor: "+2"));
            }
        }
        for (int i = 0; i < 4; i++) {
            mazo.add(new CartaComodin(valor: "Color"));
            mazo.add(new CartaComodin(valor: "+4"));
        }
        Collections.shuffle(mazo);
    }
}

```

```

    public Carta robar() {
        if (mazo.isEmpty()) {
            if (descarte.size() <= 1) return null;
            Carta cima = descarte.pop();
            mazo.addAll(descarte);
            descarte.clear();
            descarte.push(cima);
            Collections.shuffle(mazo);
        }
        return mazo.pollFirst();
    }

    public void devolverAlFondo(Carta c) {
        mazo.addLast(c);
    }

    public void dejarEnMesa(Carta c) { descarte.push(c); }
    public Carta verCima() { return descarte.peek(); }
}

```


Las mejores en esta clase fue la implementación de estructuras de datos especializadas, reemplazando una lista genérica por una LinkedList para el mazo y un Stack para el descarte, además de integrar polimorfismo al construir la baraja instanciando directamente las nuevas subclases como CartaNumerica o CartaAccion. También se añadió una mecánica de reciclaje integral donde el método robar() detecta automáticamente si el mazo se queda sin cartas para tomar el descarte y volverlo a barajar. Todo esto se hizo porque utilizar un Stack modela de forma mucho más realista la física de apilar cartas en la mesa bajo el principio LIFO (el último en entrar, primero en salir). Asimismo, este sistema de reciclaje garantiza la continuidad de tu simulador para que la partida no se rompa por falta de cartas, logrando al mismo tiempo un encapsulamiento limpio que hará mucho más legible la programación del ciclo principal o de la inteligencia artificial de la CPU al utilizar métodos dedicados como dejarEnMesa() en lugar de manipular las colecciones manualmente.

Evidencia código versión 2

```
class Jugador {
    Cadena nombre ;
    List < Carta > mano = new ArrayList <> ( ) ;

    public Jugador ( String nombre ) {
        este . nombre = nombre ;
    }

    void recibir ( Carta c ) {
        if ( c != null ) mano . add ( c ) ;
    }

    void verMano ( ) {
        Sistema . afuera . println ( " \n Mano de " + nombre + " ( " + mano . size ( ) + " cartas): " ) ;
        para ( int i = 0 ; i < mano . size ( ) ; i += 5 ) {
            int fin = Math.min ( i + 5 , mano.size ( ) ) ;
            mostrarFila ( i , fin ) ;
        }
    }

    private void mostrarFila ( int inicio , int fin ) {
        // Alineación corregida de los índices centrados sobre las cartas
        for ( int i = inicio ; i < fin ; i ++ ) {
            System.out.printf ( " [ % d ] " , i ) ;
        }
        Sistema.out.println ( ) ;

        for ( int linea = 0 ; linea < 7 ; linea ++ ) {
            for ( int i = inicio ; i < fin ; i ++ ) {
                String [ ] d = mano.get ( i ) .dibujar ( ) ;
                System.out.print ( d [ linea ] + " " ) ;
            }
            Sistema.out.println ( ) ;
        }
    }
}
```

Evidencia código versión 3

```

abstract class Jugador {
    protected String nombre;
    protected List<Carta> mano = new ArrayList<>();

    public Jugador(String nombre) { this.nombre = nombre; }
    public void recibir(Carta c) { if (c != null) mano.add(c); }
    public String getNombre() { return nombre; }
    public List<Carta> getMano() { return mano; }
    public boolean noTieneCartas() { return mano.isEmpty(); }

    public abstract void ejecutarTurno(Consola ui, GestorReglas reglas);
    public abstract String elegirColor(Consola ui);
}

```

```

class Humano extends Jugador {
    public Humano(String n) { super(n); }

    @Override
    public void ejecutarTurno(Consola ui, GestorReglas reglas) {
        ui.mostrarMano(nombre, mano);
        int op = ui.pedirEntero(prompt: "1. Jugar carta | 2. Robar: ", valorPorDefecto: 0);

        if (op == 2) {
            Carta robada = reglas.procesarRobo(this);
            ui.mostrarMensaje(mensaje: "Robaste:");
            for (int i = 0; i < 7; i++) ui.mostrarMensaje(robada.getLinea(i));

            String jugar = ui.pedirCadena(prompt: "¿Jugarla? (s/n): ");
            if (jugar.equalsIgnoreCase(anotherString: "s") && reglas.esJugadaValida(this, robada)) {
                reglas.jugarCarta(this, robada);
            } else {
                this.recibir(robada);
            }
        } else {
            int idx = ui.pedirEntero(prompt: "Selecciona el índice de la carta: ", -1);
            if (idx >= 0 && idx < mano.size()) {
                Carta seleccionada = mano.get(idx);
                if (reglas.esJugadaValida(this, seleccionada)) {
                    mano.remove(idx);
                    reglas.jugarCarta(this, seleccionada);
                } else {
                    ui.mostrarMensaje(mensaje: "Carta incompatible.");
                    ejecutarTurno(ui, reglas); // Recursividad para forzar acción válida
                }
            }
        }
    }
}

```

```

        ui.mostrarMensaje(mensaje: "Índice inválido.");
        ejecutarTurno(ui, reglas);
    }
}

@Override
public String elegirColor(Consola ui) {
    int op = ui.pedirEntero(prompt: "Elige nuevo color: 1.Rojo 2.Verde 3.Azul 4.Amarillo: ", valorPorDefecto: 1);
    String[] c = {"Rojo", "Verde", "Azul", "Amarillo"};
    return c[Math.max(a: 0, Math.min(op - 1, b: 3))];
}
}

```

```

class CPU extends Jugador {
    public CPU(String n) { super(n); }

    @Override
    public void ejecutarTurno(Consola ui, GestorReglas reglas) {
        Carta cartaAJugar = null;
        for (Carta c : mano) {
            if (reglas.esJugadaValida(this, c)) {
                cartaAJugar = c;
                break;
            }
        }

        if (cartaAJugar != null) {
            mano.remove(cartaAJugar);
            String v = cartaAJugar.getValor().equals(anObject: "Color") ? "comodin" : cartaAJugar.getValor();
            ui.mostrarMensaje(nombre + " jugó: " + v + " " + cartaAJugar.getColor());
            reglas.jugarCarta(this, cartaAJugar);
        } else {
            ui.mostrarMensaje(nombre + " no tiene cartas válidas. Roba y pasa.");
            this.recibir(reglas.procesarRobo(this));
        }
    }
}

```

de la Ciudad de México

NADA HUMANO ME ES AJENO

```

@Override
public String elegirColor(Consola ui) {
    Map<String, Integer> conteo = new HashMap<>();
    String[] colores = {"Rojo", "Verde", "Azul", "Amarillo"};
    for(String col : colores) conteo.put(col, value: 0);

    for (Carta c : mano) {
        if (!c.getColor().equals(anObject: "Negro")) {
            conteo.put(c.getColor(), conteo.get(c.getColor()) + 1);
        }
    }

    String mejorColor = colores[0];
    int max = -1;
    for (String col : colores) {
        if (conteo.get(col) > max) {
            max = conteo.get(col);
            mejorColor = col;
        }
    }

    ui.mostrarMensaje(nombre + " ha elegido el color: " + mejorColor);
    return mejorColor;
}
}

```

Lo que se mejoró en la clase Jugador de tu simulador de UNO fue la evolución de una clase concreta monolítica escrita en un tipo de pseudocódigo (versión 2) hacia una verdadera arquitectura orientada a objetos utilizando Java puro (versión 3). Para lograrlo, transformaste al jugador en una clase abstracta base y creaste subclases especializadas para Humano y CPU, delegando además las tareas visuales y normativas a componentes externos como Consola y GestorReglas. Esto se hizo porque, al aplicar la herencia y el principio de responsabilidad única, lograste desacoplar la interfaz gráfica de la lógica interna del jugador; además, esta separación te permitió programar una inteligencia artificial real y autónoma para la CPU, dándole la capacidad de tomar decisiones estratégicas, como analizar y elegir el color más abundante en su mano. En definitiva, realizaste estos cambios estructurales para que el código de tu proyecto sea mucho más limpio, profesional,

escalable y fácil de mantener sin el riesgo de romper el juego entero al agregar nuevas funciones.

Evidencia del código 2

```
clase Juego {
    Lista < Jugador > jugadores = new ArrayList <> ( );
    Baraja mazo = nuevo Baraja ( );
    Carta mesa ;
    Cadena colorActual ;
    int turno = 0 ;
    Scanner sc = new Scanner ( System . in ) ;

    registrador nulo ( String nombre ) {
        // Modalidad Humano vs Computadora
        jugadores . add ( nuevo Jugador ( nombre ) ) ;
        jugadores . agregar ( nuevo Jugador ( " CPU " ) ) ;
    }

    void repartir ( ) {
        para ( Jugador j : jugadores )
            para ( int i = 0 ; i < 7 ; i ++ ) j . recibir ( mazo . repartir ( ) ) ;

        boolean cartaValida = false ;
        mientras ( ! cartaVálida ) {
            mesa = mazo . repartir ( ) ;
            if ( mesa . valor . matches ( " \\ d " ) && ! mesa . color . igual ( " Negro " ) ) {
                cartaValida = verdadero ;
            } demás {
                mazo . cartas . agregar ( 0 , mesa ) ;
            }
        }
        colorActual = mesa . color ;
    }

    booleano esValida ( Carta c ) {
        return c . color . equals ( colorActual ) || c . valor . equals ( mesa . valor ) || c . color . equals ( " Negro " )
    }
}
```

```
void aplicarEfecto ( Carta c ) {
    // Regla: Reversa con 2 jugadores funciona como Salto
    if ( c . valor . es igual a ( " salto " ) || c . valor . es igual a ( " rev " ) ) {
        avanzar ( ) ;
        Sistema . afuera . println ( " ¡El siguiente jugador pierde su turno! " ) ;
    } else if ( c . valor . equals ( " +2 " ) ) {
        avanzar ( ) ;
        Jugador j = jugadores . obtener ( turno ) ;
        j . recibir ( mazo . repartir ( ) ) ; j . recibir ( mazo . repartir ( ) ) ;
        Sistema . afuera . println ( j.nombre + " roba 2 cartas y pierde turno. " ) ;
    } else if ( c . valor . equals ( " +4 " ) ) {
        avanzar ( ) ;
        Jugador j = jugadores . obtener ( turno ) ;
        para ( int i = 0 ; i < 4 ; i ++ ) j . recibir ( mazo . repartir ( ) ) ;
        Sistema . afuera . println ( j.nombre + " roba 4 cartas y pierde turno. " ) ;
    }
}

void avanzar ( ) {
    turno = ( turno + 1 ) % jugadores . tamaño ( ) ;
}
}
```

Evidencia código versión 3

```

class Juego {
    private List<Jugador> jugadores = new ArrayList<>();
    private Baraja baraja = new Baraja();
    private Consola ui = new Consola();
    private ManejadorTurnos turnos;
    private GestorReglas reglas;

    public Juego(String nombreH, int cpus) {
        jugadores.add(new Humano(nombreH));
        String[] nombresCPUs = {"Pablo", "Tania", "Maria"};
        for (int i = 0; i < cpus && i < nombresCPUs.length; i++) {
            jugadores.add(new CPU(nombresCPUs[i]));
        }

        turnos = new ManejadorTurnos(jugadores.size());
        reglas = new GestorReglas(baraja, turnos, jugadores, ui);

        // Repartir cartas iniciales
        for (Jugador j : jugadores) {
            for (int i = 0; i < 7; i++) j.recibir(baraja.robar());
        }

        establecerCartaInicial();
    }
}

```

```

private void establecerCartaInicial() {
    Carta inicio;
    // La carta inicial debe ser estrictamente numérica según el reglamento
    do {
        inicio = baraja.robar();
        if (!(inicio instanceof CartaNumerica)) {
            baraja.devolverAlFondo(inicio);
        }
    } while (!(inicio instanceof CartaNumerica));

    baraja.dejarEnMesa(inicio);
    reglas.setColorActual(inicio.getColor());

    ui.mostrarMensaje(mensaje: "\n--- ¡LA PRIMERA CARTA ES! ---");
    for (int i = 0; i < 7; i++) ui.mostrarMensaje(inicio.getLinea(i));
}

```



```

public void iniciar() {
    while (true) {
        if (turnos.debeSaltar()) {
            ui.mostrarMensaje(">>> SE SALTA EL TURNO DE " + jugadores.get(turnos.getTurnoActual()).getNombre() + " <<<");
            turnos.avanzar();
            continue;
        }

        Jugador actual = jugadores.get(turnos.getTurnoActual());
        ui.mostrarMensaje("\nTURNO ACTUAL: " + actual.getNombre());
        ui.mostrarMesa(baraja.verCima(), reglas.getColorActual());

        actual.ejecutarTurno(ui, reglas);

        // Verificar si debe cantar UNO
        if (actual.getMano().size() == 1) {
            if (actual instanceof Humano) {
                String respuesta = ui.pedirCadena(prompt: "\n¡ESCRIBE 'UNO'!");
                if (!respuesta.equalsIgnoreCase(anotherString: "UNO")) {
                    reglas.penalizarPorNoDecirUno(actual);
                }
            } else {
                ui.mostrarMensaje("\n" + actual.getNombre() + " grita UNO!");
            }
        }
    }
}

```

```

        // Condición de victoria
        if (actual.noTieneCartas()) {
            ui.mostrarMensaje(mensaje: "\n*****");
            ui.mostrarMensaje("¡EL GANADOR ES: " + actual.getNombre() + "!");
            ui.mostrarMensaje(mensaje: "*****");
            break;
        }

        turnos.avanzar();
    }
    ui.cerrar();
}

```

Lo que se mejoró principalmente en la clase Juego fue la transformación de un código monolítico y rígido escrito en una especie de pseudocódigo hacia una sólida arquitectura orientada a objetos utilizando Java puro. Específicamente, extrajiste todas las responsabilidades acumuladas en la clase principal y las dividiste en componentes especializados como Consola, ManejadorTurnos y GestorReglas, además de encapsular los atributos haciéndolos privados. A la par, integraste un flujo de juego mucho más robusto que ahora incluye la validación estricta para que la carta inicial sea numéricamente correcta, la regla obligatoria de gritar "UNO" y la detección explícita del ganador. Todo esto se hizo porque, al aplicar el Principio de Responsabilidad Única, lograste un desacoplamiento total entre la lógica, las normativas y la interfaz, protegiendo el estado del juego contra modificaciones

accidentales. En definitiva, realizaste estos cambios para garantizar que tu simulador sea fiel a las reglas oficiales y para que el código resultante sea completamente escalable, profesional y fácil de mantener.

Evidencia del código 2

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Introduce tu nombre:");  
        String nombre = sc.nextLine();  
  
        Juego juego = new Juego();  
        juego.registrar(nombre);  
        juego.repartir();  
  
        while (true) {  
            Jugador actual = juego.jugadores.get(juego.turno);  
            System.out.println("\n--- CARTA EN MESA ---");  
            for (String l : juego.mesa.dibujar()) System.out.println(l);  
            System.out.println("Color actual: " + juego.colorActual);  
  
            if (!actual.nombre.equals("CPU")) {  
                actual.verMano();  
                System.out.println("1. Jugar | 2. Robar");  
                int op = leerOpcion(sc, 1, 2);  
  
                if (op == 1) {  
                    System.out.println("Indice de carta:");  
                    int idx = leerOpcion(sc, 0, actual.mano.size() - 1);  
                    Carta elegida = actual.mano.get(idx);  
  
                    if (juego.esValida(elegida)) {  
                        procesarJugada(juego, actual, elegida, sc);  
                    } else {  
                        System.out.println("Carta inválida. Elige otra o roba.");  
                        continue;  
                    }  
                } else {  
                    Carta robada = juego.mazo.repartir();  
                    System.out.println("\nRobaste:");  
                    for (String l : robada.dibujar()) System.out.println(l);  
                }  
            }  
        }  
    }  
}
```

```

        System.out.println("¿Quieres jugarla? (s/n)");
        if (sc.nextLine().equalsIgnoreCase("s")) {
            if (juego.esValida(robada)) {
                procesarJugada(juego, actual, robada, sc);
            } else {
                // Regla: Si roba y La carta no es válida -> pierde turno
                System.out.println("No se puede jugar. Pierdes turno.");
                actual.recibir(robada);
            }
        } else {
            System.out.println("Carta guardada. Pierdes turno.");
            actual.recibir(robada);
        }
    }

    // Regla "UNO": Si jugador tiene 1 carta debe decir "UNO"
    if (actual.mano.size() == 1) {
        System.out.print("¡Escribe UNO! ");
        if (!sc.nextLine().equalsIgnoreCase("UNO")) {
            System.out.println("Penalización +2 por no decir UNO.");
            actual.recibir(juego.mazo.repartir());
            actual.recibir(juego.mazo.repartir());
        }
    }
} else {
    Carta paraJugar = null;
    for (Carta c : actual.mano) {
        if (juego.esValida(c)) { paraJugar = c; break; }
    }

    if (paraJugar != null) {
        System.out.println("\nCPU jugó: " + paraJugar.color + " " + paraJugar.valor);
        procesarJugada(juego, actual, paraJugar, sc);
    } else {
        System.out.println("\nCPU roba y pasa.");
        actual.recibir(juego.mazo.repartir());
    }
}
}

```

Evidencia del código versión 3

```

public class Main {
    public static void main(String[] args) {
        Consola uiInicial = new Consola();
        uiInicial.mostrarMensaje(mensaje: "=====");
        uiInicial.mostrarMensaje(mensaje: "      BIENVENIDO AL JUEGO UNO      ");
        uiInicial.mostrarMensaje(mensaje: "=====");

        String n = uiInicial.pedirCadena(prompt: "Nombre: ");
        int c = uiInicial.pedirEntero(prompt: "CPUs (1-3): ", valorPorDefecto: 1);

        new Juego(n, Math.max(a: 1, Math.min(a: 3, c))).iniciar();
    }
}

```

En la clase Main se mejoró drásticamente la estructura al eliminar toda la lógica del juego, como el ciclo de turnos y las validaciones, que antes saturaban este archivo. Lo que hiciste fue reducir el método principal a su mínima expresión para que actúe únicamente como el punto de entrada, delegando la interacción con el usuario a un

componente centralizado llamado Consola y encapsulando el arranque limpio en la instrucción `new Juego(...).iniciar()`. Todo esto se hizo porque era fundamental aplicar el principio de responsabilidad única para que tu código dejara de ser un script monolítico. Al separar estas responsabilidades, lograste desacoplar la lógica interna de la consola, haciendo al programa mucho más robusto frente a errores del usuario al limitar matemáticamente la configuración inicial de los oponentes. Esta importante evolución arquitectónica transforma tu simulador en un sistema profesional, altamente legible y totalmente preparado para el futuro, facilitando enormemente su mantenimiento o la posible integración de una interfaz gráfica sin alterar las reglas del juego.

```
class Consola {
    private Scanner sc = new Scanner(System.in);

    public void mostrarMensaje(String mensaje) {
        System.out.println(mensaje);
    }

    public void mostrarMesa(Carta cima, String colorActual) {
        System.out.println(x: "\n-----");
        System.out.println(x: "CARTA EN MESA:");
        for (int i = 0; i < 7; i++) System.out.println(cima.getLinea(i));

        String valMesa = cima.getValor().equals(anObject: "Color") ? "comodin" : cima.getValor();
        System.out.println("ESTADO -> Color: " + colorActual + " | Valor: " + valMesa);
    }

    public void mostrarMano(String nombre, List<Carta> mano) {
        System.out.println("\nMano de " + nombre + " (" + mano.size() + " cartas):");
        for (int i = 0; i < mano.size(); i += 7) {
            int fin = Math.min(i + 7, mano.size());
            for (int j = i; j < fin; j++) System.out.printf(format: "    [%d]    ", j);
            System.out.println();
            for (int l = 0; l < 7; l++) {
                for (int j = i; j < fin; j++) System.out.print(mano.get(j).getLinea(l) + " ");
                System.out.println();
            }
        }
    }
}
```

```
class ManejadorTurnos {
    private int turnoActual = 0; // Índice en la lista de jugadores
    private int direccion = 1;    // 1 horario, -1 antihorario
    private boolean saltarProximo = false;
    private int cantidadJugadores;

    public ManejadorTurnos(int cantidadJugadores) {
        this.cantidadJugadores = cantidadJugadores;
    }

    public int getTurnoActual() { return turnoActual; }

    public int getSiguienteTurno() {
        return (turnoActual + (direccion + cantidadJugadores)) % cantidadJugadores;
    }

    public void avanzar() {
        turnoActual = getSiguienteTurno();
    }

    public void activarSalto() {
        saltarProximo = true;
    }

    public boolean debeSaltar() {
        if (saltarProximo) {
            saltarProximo = false;
            return true;
        }
        return false;
    }
}
```

```
    public void invertirDireccion() {
        if (cantidadJugadores == 2) {
            activarSalto(); // En 1v1, el reverso actúa como salto
        } else {
            direccion *= -1;
        }
    }
}
```

```

class GestorReglas {
    private Baraja baraja;
    private ManejadorTurnos turnos;
    private List<Jugador> jugadores;
    private Consola ui;
    private String colorActual;

    public GestorReglas(Baraja b, ManejadorTurnos t, List<Jugador> j, Consola ui) {
        this.baraja = b;
        this.turnos = t;
        this.jugadores = j;
        this.ui = ui;
    }

    public String getColorActual() { return colorActual; }
    public void setColorActual(String color) { this.colorActual = color; }

    public boolean esJugadaValida(Jugador j, Carta c) {
        if (c.getValor().equals(anObject: "+4")) {
            // Regla oficial: Solo puedes tirar +4 si no tienes el color actual
            for (Carta m : j.getMano()) {
                if (m.getColor().equals(colorActual)) return false;
            }
        }
        return c.esCompatible(baraja.verCima(), colorActual) || c.getColor().equals(anObject: "Negro");
    }
}

```

```

    public void jugarCarta(Jugador j, Carta c) {
        baraja.dejarEnMesa(c);
        if (!c.getColor().equals(anObject: "Negro")) {
            colorActual = c.getColor();
        }
        c.aplicarEfecto(turnos, this);
    }

    public Carta procesarRobo(Jugador j) {
        return baraja.robar();
    }

    public void penalizarSiguiete(int cantidadCartas) {
        int indiceVictima = turnos.getSiguienteTurno();
        Jugador victima = jugadores.get(indiceVictima);

        for (int i = 0; i < cantidadCartas; i++) {
            victima.recibir(baraja.robar());
        }

        ui.mostrarMensaje("¡" + victima.getNombre() + " roba " + cantidadCartas + " cartas y pierde su turno!");
        turnos.activarSalto();
    }

    public void solicitarCambioColor() {
        Jugador activo = jugadores.get(turnos.getTurnoActual());
        colorActual = activo.elegirColor(ui);
    }
}

```

```
public void penalizarPorNoDecirUno(Jugador j) {  
    ui.mostrarMensaje(mensaje: "Penalización +2 cartas por no decir UNO.");  
    j.recibir(baraja.robar());  
    j.recibir(baraja.robar());  
}  
}
```

Al momento de analizar la evolución del simulador de UNO, lo que se mejoró al agregar estas tres nuevas clases fue la modularización total del sistema aplicando el Principio de Responsabilidad Única, dejando atrás la versión 2 donde todo estaba mezclado. Creaste la clase Consola para aislar exclusivamente las entradas y salidas visuales, el ManejadorTurnos para encapsular la compleja matemática direccional del juego (incluyendo la regla del reverso en 1v1), y el GestorReglas para actuar como un árbitro independiente que valida jugadas estrictas como la condición oficial del comodín +4 o las penalizaciones. Esto se hizo porque estructurar el código de esta forma te permite tener un proyecto altamente cohesivo y desacoplado, algo vital en tu formación como ingeniero de software; gracias a esta arquitectura limpia, el día de mañana podrás reemplazar la consola de texto por una interfaz gráfica elaborada o incluso llevar el motor de este juego a Godot sin tener que modificar ni una sola línea de la lógica interna de tus reglas o turnos.

Conclusión:

Las mejoras estructurales implementadas en esta última versión trascienden la simple funcionalidad del juego para convertirse en una verdadera demostración de ingeniería de software aplicada. Al encapsular de manera rigurosa la lógica del negocio y desacoplarla por completo de la interfaz de texto, se ha logrado blindar el núcleo del sistema, obteniendo un código altamente cohesivo y excepcionalmente fácil de mantener. Esta arquitectura modular garantiza una escalabilidad absoluta, dejando el proyecto con una base tan sólida que está perfectamente preparado para dar el siguiente gran paso técnico, como integrarlo fluidamente a un entorno visual avanzado o incluso portar el motor del juego directamente a Godot sin tener que reescribir su funcionamiento interno.